

P1102R1

Down with `()`!

Published Proposal, 2020-11-16

This version:

<http://wg21.link/P1102R1>

Authors:

[Alex Christensen](#) (Apple)

[JF Bastien](#) (Toyota Research Institute—Advanced Development)

Audience:

CWG

Project:

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Source:

<https://github.com/achristensen07/papers/blob/master/source/P1102r1.bs>

Abstract

A proposal for removing unnecessary `()`'s from C++ lambdas.

Table of Contents

1 Revision History

2 Introduction and motivation

3 Impact

4 Wording

References

Informative References

§ 1. Revision History

EWG discussed this change as [\[EWG135\]](#) in [Lenexa](#) and voted 15 to 1 on forwarding to core. It became [\[CWG2121\]](#), discussed in [Kona](#) and needed someone to volunteer wording. This paper presents text that implements that decision, for consideration.

[\[P1102R0\]](#) was published in June 2018. It was discussed on the EWG reflector in June 2018, Nina Ranns provided feedback, and EWG chair agreed that the paper should move to CWG directly given previous polls.

P1102R1 responds to feedback about ambiguous requires-clauses from Hubert Tong.

§ 2. Introduction and motivation

Currently, C++ lambdas with no parameters do not require a parameter declaration clause. The specification even contains this language in [\[expr.prim.lambda\]](#) section 8.4.5 ¶4:

If a lambda-expression does not include a lambda-declarator, it is as if the lambda-declarator were `()`.

This allows us to omit the unused `()` in simple lambdas such as this:

```
std::string s1 = "abc";
auto withParen = [s1 = std::move(s1)] () {
    std::cout << s1 << '\n';
};

std::string s2 = "abc";
auto noSean = [s2 = std::move(s2)] { // Note no syntax error.
    std::cout << s2 << '\n';
};
```

These particular lambdas have ownership of the strings, so they ought to be able to mutate it, but `s1` and `s2` are const (because the `const` operator is declared `const` by default) so we need to add the `mutable` keyword:

```

std::string s1 = "abc";
auto withParen = [s1 = std::move(s1)] () mutable {
    s1 += "d";
    std::cout << s1 << '\n';
};

std::string s2 = "abc";
auto noSean = [s2 = std::move(s2)] mutable { // Currently a syntax error.
    s2 += "d";
    std::cout << s2 << '\n';
};

```

Confusingly, the current Standard requires the empty parens when using the `mutable` keyword. This rule is unintuitive, causes common syntax errors, and clutters our code. When compiling with clang, we even get a syntax error that indicates the compiler knows exactly what is going on:

```

example.cpp:11:54: error: lambda requires '()' before 'mutable'
auto noSean = [s2 = std::move(s2)] mutable { // Currently a syntax error.
                                ^
                                ()

1 error generated.

```

This proposal would make these parentheses unnecessary like they were before we added `mutable`. This will apply to:

- lambda template parameters
- `constexpr`
- `mutable`
- Exception specifications and `noexcept`
- attributes
- trailing return types
- `requires`

§ 3. Impact

This change will not break existing code.

§ 4. Wording

Modify Lambda expressions [**expr.prim.lambda**] as follows:

```
lambda-expression :  
    lambda-introducer lambda-parameter-listopt lambda-declaratoropt lambda-specifiers  
    compound-statement  
lambda-introducer < template-parameter-list > requires-clauseopt  
    lambda-parameter-list-and-specifiersopt lambda-declaratoropt compound-statement  
lambda-introducer :  
    [ lambda-captureopt ]  
lambda-parameter-list :  
    ( parameter-declaration-clause ).  
lambda-parameter-list-and-specifiers :  
    lambda-parameter-list lambda-specifiers  
lambda-declarator lambda-specifiers :  
    ( parameter-declaration-clause ) decl-specifier-seqopt  
    noexcept-specifieropt attribute-specifier-seqopt trailing-return-typeopt requires-  
clauseopt
```

Because parts of *lambda-declarator* have been renamed to *lambda-parameter-list* and *lambda-specifiers*, each reference to *lambda-declarator* must be updated.

Modify Function parameter scope [**basic.scope.param**] §1:

A function parameter (including one appearing in a ~~lambda-declarator~~ lambda-parameter-list) or function-local predefined variable (9.5) has function parameter scope. [...]

Modify Lambda expressions [**expr.prim.lambda.general**] §3:

In the *decl-specifier-seq* of the ~~lambda-declarator~~ lambda-specifiers, each *decl-specifier* shall be one of *mutable*, *constexpr*, or *constexpr*.

Modify Lambda expressions [**expr.prim.lambda.general**] §4:

If a *lambda-expression* does not include a ~~lambda-declarator~~ lambda-parameter-list, it is as if the ~~lambda-declarator~~ lambda-parameter-list were (). The lambda return type is `auto`, which is replaced by the type specified by the *trailing-return-type* if provided and/or deduced from return statements as described in 9.2.9.6.

Modify Closure types [**expr.prim.lambda.closure**] §2 Note 1:

This determines the set of namespaces and classes associated with the closure type (6.5.3). The parameter types of a ~~lambda-declarator~~ lambda-parameter-list do not affect these associated namespaces and classes.

Modify Closure types [**expr.prim.lambda.closure**] §3:

[...] The trailing *requires-clause* of the function call operator or operator template is the *requires-clause* following the ~~lambda-declarator~~ lambda-parameter-list, if any.

Modify Closure types [**expr.prim.lambda.closure**] §4:

[...] An *attribute-specifier-seq* in a ~~lambda-declarator~~ lambda-specifiers appertains to the type of the corresponding function call operator or operator template. [...]

Modify Closure types [**expr.prim.lambda.closure**] §4 Note 3:

Names referenced in the ~~lambda-declarator~~ lambda-parameter-list are looked up in the context in which the *lambda-expression* appears.

Modify Captures [**expr.prim.lambda.capture**] §5:

If an *identifier* in a *simple-capture* appears as the *declarator-id* of a parameter of the ~~lambda-declarator~~ lambda-parameter-list's *parameter-declaration-clause*, the program is ill-formed.

Modify Default arguments [**dcl.fct.default**] §3:

A default argument shall be specified only in the *parameter-declaration-clause* of a function declaration or ~~lambda-declarator~~ lambda-parameter-list or in a *template-parameter* (13.2); in the latter case, the *initializer-clause* shall be an *assignment-expression*. [...]

Modify Name resolution [**temp.res.general**] §5.2.5:

parameter-declaration in a ~~*lambda-declarator*~~ *lambda-parameter-list* or *requirement-parameter-list*, unless that *parameter-declaration* appears in a default argument, or

§ References

§ Informative References

[CWG2121]

EWG. More flexible lambda syntax. 6 May 2015. drafting. URL: <https://wg21.link/cwg2121>

[EWG135]

Herb Sutter. [tiny] Mutable is part of a lambda-declarator, so when a lambda is mutable, the parentheses aren't optional. New. URL: <https://wg21.link/ewg135>

[P1102R0]

Alex Christensen, JF Bastien. Down with ()!. 20 June 2018. URL: <https://wg21.link/p1102r0>